SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5117

NAME: G. BALASUBRAMANIAN

REG.NO:192521001

DEPARTMENT: B.TECH. IT

ASSIGNMENT: C02 – AT2

FACULTY NAME: DR. R. DHARANI

1. Secure Data Transmission using Block Cipher

A. Explain why this issue occurs in ECB mode.

Electronic Codebook (ECB) is the simplest mode of operation for a block cipher. In ECB mode, the plaintext is divided into fixed-size blocks (such as 128 bits in AES), and each block is encrypted **independently** using the same secret key.

The main problem with ECB is that **identical plaintext blocks always produce identical ciphertext blocks**. Since there is no relationship between one block and another, repeated patterns in the original data remain visible in the encrypted output.

Why is this a security issue?

- Financial records often contain repeated information such as account numbers, customer IDs, transaction codes, or fixed document formats.
- If the same data appears multiple times, ECB generates the same ciphertext every time.
- An attacker observing the encrypted data can identify repeated blocks and analyze the structure of the original message.
- Although the attacker may not know the exact content, they can recognize patterns and use them for statistical or cryptographic attacks.

Example

Plaintext Block	**Ciphertext (ECB)**
Salary = ₹50,000	A1B2C3D4
Salary = ₹50,000	A1B2C3D4
Salary = ₹75,000	X9Y8Z7W6

In the example above, the repeated salary value produces the same ciphertext (**A1B2C3D4**), revealing that the same information appears more than once.

Disadvantages of ECB

- Reveals patterns in encrypted data.
- Does not hide repeated information.
- Vulnerable to pattern analysis attacks.
- Not suitable for sensitive data such as banking, medical, or financial information.

B. Suggest a more secure block cipher mode and justify your choice.

A more secure mode for this scenario is **Cipher Block Chaining (CBC)** mode.

How CBC Works

In CBC mode:

1. Before encrypting a plaintext block, it is **XORed with the previous ciphertext block**.
2. The first plaintext block is XORed with a random **Initialization Vector (IV)**.
3. The result is then encrypted using the secret key.

This chaining process ensures that the encryption of each block depends on all previous blocks.

Advantages of CBC

- Identical plaintext blocks produce different ciphertext blocks.
- Prevents attackers from identifying repeated patterns.
- Provides better confidentiality than ECB.
- Suitable for encrypting financial records, confidential documents, and secure communications.

Example

Plaintext Block	**Ciphertext (CBC)**
Salary = ₹50,000	P8Q7R6S5
Salary = ₹50,000	M3N2O1P9

Although the plaintext is the same, the ciphertext is different because each block depends on the previous ciphertext and the Initialization Vector (IV).

Comparison of ECB and CBC

Feature	ECB	CBC
Encryption Method	Blocks encrypted independently	Each block depends on the previous ciphertext block
Repeated Plaintext	Produces identical ciphertext	Produces different ciphertext
Pattern Hiding	No	Yes
Security Level	Low	High
Suitable for Financial Data	No	Yes

Conclusion

ECB mode is not suitable for transmitting repeated financial data because it reveals patterns by producing the same ciphertext for identical plaintext blocks. This allows attackers to analyze encrypted messages and infer information about the data. **CBC mode** is a much better choice because it uses an **Initialization Vector (IV)** and chains each block with the previous ciphertext, ensuring that identical plaintext blocks generate different ciphertexts. As a result, CBC provides stronger confidentiality and is more appropriate for secure financial data transmission.

2. Choosing an Encryption Algorithm for Banking System

A. Identify the most suitable algorithm for this scenario.

The **most suitable encryption algorithm is Blowfish**.

B. Justify your choice based on security strength and efficiency.

A banking application requires **high security** to protect sensitive customer information and **moderate performance** to process transactions efficiently. Among **DES, 3-DES, and Blowfish**, Blowfish offers the best balance of security and speed.

1. DES (Data Encryption Standard)

- DES uses a **56-bit key**, which is too small by modern security standards.
- It is vulnerable to **brute-force attacks**, where attackers can try every possible key until the correct one is found.
- Although DES is fast, it no longer provides adequate protection for banking applications.
- Therefore, DES is **not recommended** for securing customer transactions.

2. 3-DES (Triple DES)

- 3-DES improves security by applying the DES algorithm **three times** with different keys.
- It provides much stronger security than DES.
- However, because it performs encryption three times, it is **significantly slower**.
- This slower performance can reduce transaction processing speed in banking systems.
- While secure, it is less efficient for modern applications.

3. Blowfish

- Blowfish is a **64-bit block cipher** that supports a **variable key size from 32 to 448 bits**.
- It offers **strong security** due to its long key length, making brute-force attacks extremely difficult.
- Blowfish is **faster than 3-DES** because it performs encryption only once while still providing strong protection.

Comparison

Feature	DES	3-DES	Blowfish
Key Size	56 bits	112/168 bits	32–448 bits
Security	Low	High	High
Speed	Fast	Slow	Fast
Brute-force Resistance	Poor	Good	Excellent
Suitable for Banking	No	Yes (but slow)	**Yes (Best Choice)**

Conclusion

For a banking application that requires **high security and moderate performance**, **Blowfish** is the most suitable algorithm. It provides stronger security than DES, is faster than 3-DES, supports large key sizes, and efficiently encrypts customer transactions. Therefore, Blowfish offers the best balance between **security strength and performance** among the given options.

3. Key Exchange in an Unsecured Network

A. Explain how the Diffie-Hellman key exchange works in this scenario.

Diffie-Hellman Key Exchange (DH) is a cryptographic method that allows two users to establish a **shared secret key** over an insecure public network without directly transmitting the secret key.

Steps Involved

1. **Choose Public Values**
    - Both users agree on two public numbers:
        - A large prime number (**P**)
        - A generator (**G**)
    - These values are public and can be known by everyone.
2. **Generate Private Keys**
    - User A selects a private key **a**.
    - User B selects a private key **b**.
    - These private keys are kept secret.
3. **Compute Public Keys**
    - User A computes:
      [
      A = G^ a \mod P
      ]
    - User B computes:
      [
      B = G^ b \mod P
      ]
4. **Exchange Public Keys**
    - User A sends **A** to User B.
    - User B sends **B** to User A.
    - Even if an attacker sees these values, the private keys remain secret.
5. **Generate the Shared Secret Key**
    - User A computes:
      [

K = B^ a \mod P
]

- User B computes:
  [
  K = A^ b \mod P
  ]
- Both users obtain the **same shared secret key**, which is then used to encrypt and decrypt messages.

Advantages

- The secret key is **never transmitted** over the network.
- Even on a public network, attackers cannot easily calculate the shared key because solving the discrete logarithm problem is computationally difficult.

B. Identify a possible security threat and suggest a solution to mitigate it.

Security Threat: Man-in-the-Middle (MITM) Attack

The biggest weakness of the basic Diffie-Hellman protocol is the **Man-in-the-Middle (MITM) attack**.

How the Attack Works

- An attacker intercepts the public keys exchanged between User A and User B.
- The attacker sends their own public key to each user instead of forwarding the original one.
- User A unknowingly establishes a secret key with the attacker.
- User B also establishes a different secret key with the attacker.
- The attacker can decrypt, modify, and re-encrypt messages before sending them to the other user, while both users believe they are communicating securely.

Solution to Mitigate the Attack

The best solution is to **authenticate the exchanged public keys**.

This can be achieved by:

- **Digital Certificates** issued by a trusted Certificate Authority (CA).
- **Digital Signatures**, where each user signs their public key before sending it.
- Using authenticated protocols such as **TLS/SSL**, which combine Diffie-Hellman with certificate-based authentication.

Authentication ensures that the received public key actually belongs to the intended user, preventing attackers from impersonating either party.

Conclusion

Diffie-Hellman enables two users to securely generate a shared secret key over a public network without transmitting the key itself. However, the protocol is vulnerable to **Man-in-the-Middle attacks** if the exchanged public keys are not authenticated. Using **digital certificates, digital signatures, or TLS/SSL** effectively prevents this attack and ensures secure communication.

4. Public Key Cryptography for Email Security

A. Describe how RSA can be used for encryption and decryption in this scenario.

RSA (Rivest–Shamir–Adleman) is a public key cryptographic algorithm used to provide secure communication. In an email system, each user has two keys:

- **Public Key** – Shared openly with others.
- **Private Key** – Kept secret by the owner.

Encryption Process

1. The sender writes the email message.
2. The sender obtains the **receiver's public key**.
3. The email is encrypted using the receiver's public key.
4. The encrypted email is sent over the network.
5. Even if an attacker intercepts the email, it cannot be read without the receiver's private key.

Decryption Process

1. The receiver receives the encrypted email.
2. The receiver uses their **private key** to decrypt the message.
3. The original email is recovered and can be read.

Example

- Alice wants to send a confidential email to Bob.
- Alice encrypts the email using **Bob's public key**.
- Bob decrypts the email using **his private key**.
- Since only Bob possesses the private key, only he can read the message.

Advantages of RSA in Email Security

- Ensures **confidentiality** by preventing unauthorized access.
- Only the intended recipient can decrypt the email.
- Public keys can be shared safely without compromising security.

B. Explain how key distribution and management play a role in maintaining security.

Key Distribution

Key distribution is the process of securely providing public keys to users.

- Each user publishes their **public key** so others can encrypt emails.
- Public keys should be distributed through a **trusted source**, such as a Public Key Infrastructure (PKI) or digital certificates.
- This ensures that users receive the correct public key and not a fake one created by an attacker.

Key Management

Key management involves generating, storing, updating, and protecting cryptographic keys throughout their lifecycle.

- **Protecting Private Keys:** Private keys must remain confidential and should never be shared.
- **Key Storage:** Private keys should be stored securely using encrypted storage or hardware security modules (HSMs).
- **Key Renewal:** Keys should be replaced periodically to reduce security risks.
- **Key Revocation:** If a private key is compromised, the corresponding public key certificate should be revoked immediately and replaced with a new key pair.
- **Backup and Recovery:** Secure backups of private keys help prevent permanent data loss.

Importance of Key Distribution and Management

- Prevents attackers from impersonating legitimate users.
- Protects against unauthorized access to encrypted emails.
- Ensures only intended recipients can decrypt messages.
- Maintains the confidentiality, integrity, and authenticity of email communication.

Conclusion

RSA provides secure email communication by encrypting messages with the **recipient's public key** and decrypting them with the **recipient's private key**. Effective **key distribution** ensures users receive authentic public keys, while proper **key management** protects private keys, supports key renewal and revocation, and maintains the overall security of the email system.

5. Cryptography for IoT Devices

A. Analyze the constraints of IoT devices.

IoT (Internet of Things) devices, such as smart home sensors, smart locks, cameras, and thermostats, have limited hardware resources. Therefore, the cryptographic algorithm used must be lightweight and efficient.

Constraints of IoT Devices

1. Limited Processing Power

- IoT devices use low-power microcontrollers with limited CPU capabilities.
- Complex encryption algorithms can increase processing time and reduce device performance.

2. Limited Memory

- Most IoT devices have small RAM and storage capacity.
- Cryptographic algorithms requiring large keys or extensive computations consume more memory.

3. Low Power Consumption

- Many IoT devices operate on batteries.
- Heavy cryptographic operations consume more energy and reduce battery life.

4. Limited Bandwidth

- IoT devices often communicate over low-speed wireless networks.
- Large encryption keys increase communication overhead and transmission time.

5. Security Requirements

- Despite limited resources, IoT devices must ensure:
    - **Confidentiality** – Prevent unauthorized access to data.
    - **Integrity** – Protect data from modification.
    - **Authentication** – Verify the identity of communicating devices.

B. Recommend a suitable cryptographic approach and justify your answer.

The **recommended cryptographic approach is Elliptic Curve Cryptography (ECC).**

Justification

ECC provides the same level of security as RSA while using much smaller key sizes.

Advantages of ECC

1. Smaller Key Size

- ECC achieves strong security with much smaller keys.
- Example:
    - RSA **3072-bit** ≈ ECC **256-bit** (similar security level).

2. Faster Computation

- Smaller keys require fewer mathematical operations.
- This results in faster encryption, decryption, and key generation.

3. Lower Power Consumption

- ECC performs fewer calculations than RSA.
- This reduces battery usage, making it ideal for battery-powered IoT devices.

4. Lower Memory Usage

- ECC requires less storage for keys and certificates.
- It is suitable for devices with limited RAM and flash memory.

5. Better Performance

- Faster cryptographic operations improve communication speed and overall system efficiency.

Comparison of RSA and ECC

Feature	**RSA**	**ECC**
Key Size	Large (2048–3072 bits)	Small (256–521 bits)
Security	High	High (same security with smaller keys)
Processing Speed	Slower	Faster
Memory Requirement	High	Low
Power Consumption	High	Low
Suitable for IoT	No	**Yes**

Conclusion

IoT devices have **limited processing power, memory, battery life, and bandwidth**, making lightweight cryptographic algorithms essential. **Elliptic Curve Cryptography (ECC)** is the best choice because it provides **strong security with smaller key sizes**, requires **less memory**, consumes **less power**, and performs **faster** than RSA. Therefore, ECC is the most suitable cryptographic approach for secure communication in smart home IoT systems.